8051 Microcontroller

Chapter 3.2: Subroutine Calls — LCALL, ACALL, and RET

1. Introduction

Call is an instruction in assembly language that temporarily transfers program control to a subroutine, function, method, or procedure.

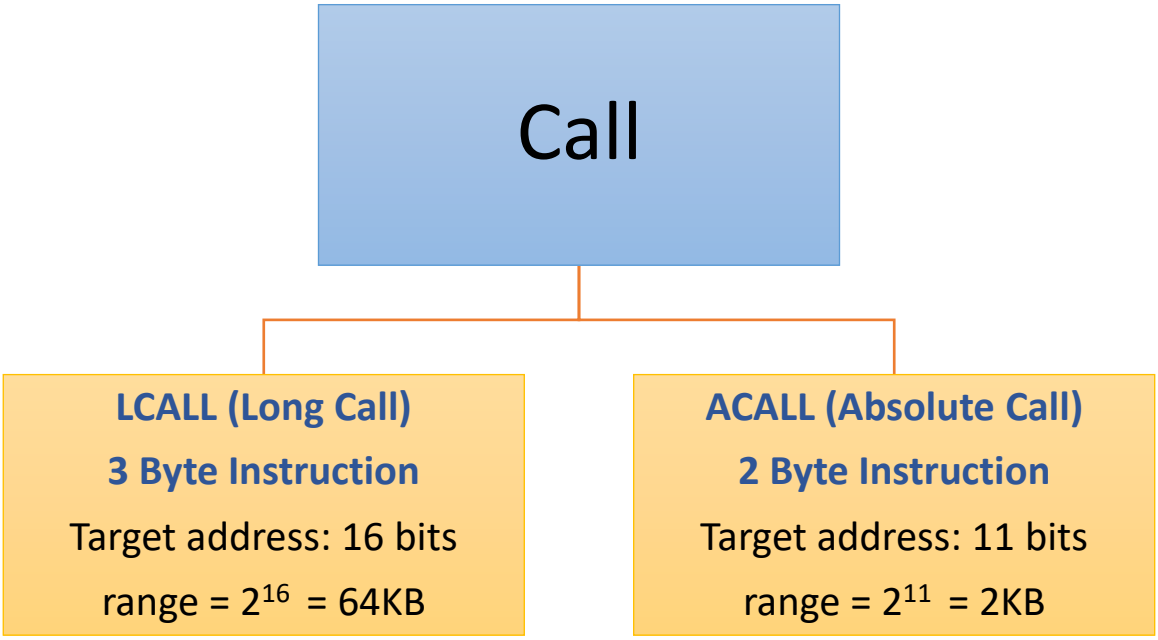
Subroutine (function) is block of code that performs a task, such as Sum, Sort, and Average.

The CALL instruction is used to jump to a subroutine, which later returns control back to the instruction immediately following where it was called from.

The 8051 provides two call instructions,

1- LCALL (Long Call)

2- ACALL (Absolute Call)



2. LCALL — Long Call

- 3-byte instruction.
- First byte: opcode.
- Second and third bytes: the target address — 2 bytes (16-bit).

Target address = 16 bits = 2 bytes.

$$2^{16} = 2^{6} \times 2^{10} = 64 \text{ KB}$$

Because the target address is a full 16 bits, LCALL can call a subroutine located anywhere in the full 64 KB address space (ROM).

3. ACALL — Absolute Call

- 2-byte instruction.
- 5 bits of the instruction form the opcode.
- The remaining 11 bits are used as the target address.

```
Target address = 11 bits  →  2¹¹ = 2 KB
```

ACALL's reach is therefore limited to the same 2 KB block of memory as the instruction following it — the same page restriction that applies to AJMP.

4. What Happens When LCALL Executes

On reset (power-on), the program counter always starts at **PC = 0000H**. Consider the following program, where the main code calls two subroutines:

```
address         ORG   0000H
                :
                :
                LCALL SQR1
                INC   A
                :
                ACALL SQR2
                :
                LCALL SQR1
                MOV   R2, A
                :
                SJMP  $

                ORG   250H
SQR1:           MOV   A, R1          ; ┐
                ADD   A, R2          ; │  subroutine 1
                RET                  ; ┘

SQR2:           MOV   B, A           ; ┐
                :                    │  subroutine 2
                RET                  ; ┘
                END
```

The Two Steps of LCALL

- 1. PUSH the current content of PC (the address of the instruction right after LCALL) onto the stack.

  SP = SP + 2
- 2. Load PC with the target address of the subroutine (e.g., PC = 0250H for SQR1).

RET — Return to the Main Program

RET pops two bytes from the stack back into PC, restoring exactly the return address that LCALL pushed, so execution resumes right after the call instruction in the main program.

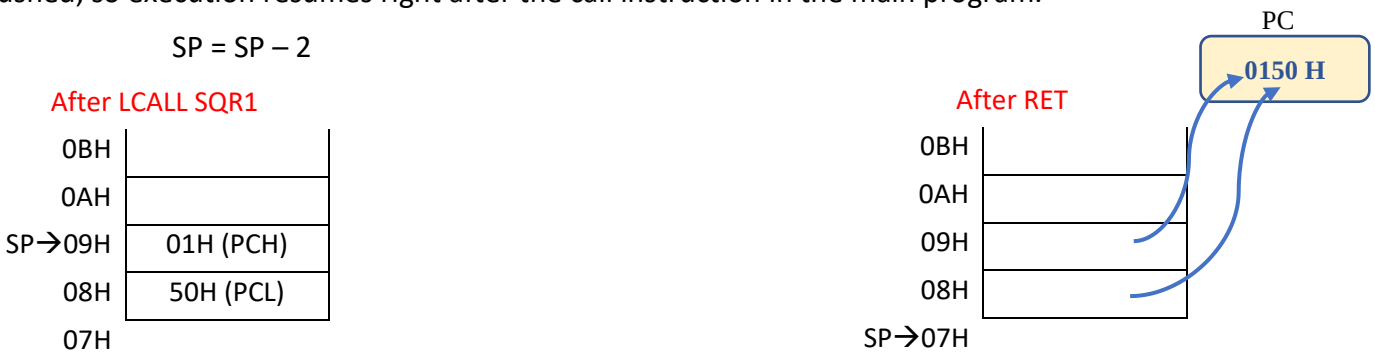


Figure 2. LCALL pushes the return address onto the stack and jumps to the subroutine; RET later pops that address back into PC.

Stack pointer change:
- LCALL increments SP by 2 (two bytes pushed: PCH then PCL).
- RET decrements SP by 2 (two bytes popped back into PC).

5. Example — Keeping the Stack Balanced

Suppose LCALL SQR1 is executed at address 0150H (a 3-byte instruction, so the return address pushed is 0150H + 3 = 0153H), and the subroutine itself also uses the stack:

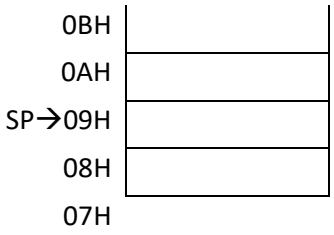
```
0150H   LCALL   SQR1
        INC     A
        :
        ORG     250H
SQR1:   MOV     R2, #80H
        PUSH    2
        :
        POP     2
        RET
```

Step-by-Step Trace

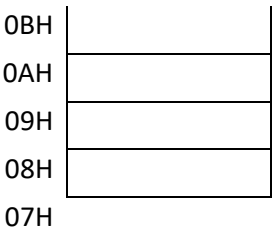
Step	SP	Stack Contents
After LCALL SQR1 (return addr. 0153H pushed)	**09H**	[08H]=53H (PCL) [09H]=01H (PCH)
After PUSH 2 (R2 = 80H pushed)	**0AH**	[08H]=53H [09H]=01H [0AH]=80H
After POP 2 (restores R2 = 80H; removes top byte)	**09H**	[08H]=53H [09H]=01H
After RET (pops PCH, PCL)	**07H**	PC = 0153H — correct return, right after LCALL

Pitfall: if the matching POP 2 were skipped (or placed after RET), the stack would be left unbalanced by one extra byte. RET would then pop the wrong two bytes — for example, treating the pushed R2 value (80H) and the PCH byte (01H) as the return address, producing PC = 8001H instead of 0153H. Execution would jump to a meaningless location and the program would effectively crash.

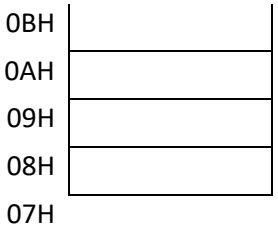
After LCALL SQR1



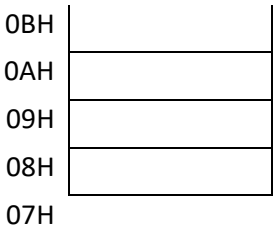
After PUSH 2



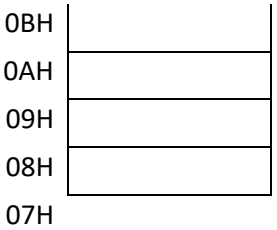
After POP 2



After RET



After RET



Golden rule: in any subroutine, the number of PUSH instructions must equal the number of POP instructions, so the stack is exactly balanced again before RET executes.

6. Instructions That Affect the Stack Pointer

The following instructions change the value of SP, and so must be balanced carefully within a subroutine:

Instruction	Effect on SP
PUSH	SP = SP + 1 (per byte pushed)
POP	SP = SP − 1 (per byte popped)
LCALL (and ACALL)	SP = SP + 2 (pushes the 2-byte return address)
RET	SP = SP − 2 (pops the 2-byte return address)
MOV SP, #data	sets SP directly to a new value (relocates the stack)